

Okapi

Installation & Integration Guide

Ask once. Ask right. Never get stuck.

A step-by-step guide to adding Okapi, the complete permission manager for Flutter, to your Android and iOS app. Written so that anyone can follow it, even if this is your first Flutter package.

Developed by

Sanket Santosh Vidya Manohar

v1.0.0

CONTENTS

What's inside

- Welcome
- Before you start
1 Add Okapi to your project
2 Set up Android
3 Set up iOS
4 Import and configure
5 Ask your first permission
6 Understand the result
7 When the user says no
8 Ask several permissions
9 Ready-made widgets
10 Make it yours
11 Analytics hooks
12 Test it
13 Check and fix with Doctor and Sync
14 Ask for features
15 Stop nagging with an ask policy
16 Keep apps alive on Indian phones
17 Protect from auto-reset
18 Add a Privacy Center
19 Test with the debug panel
20 Before you publish
- Troubleshooting
- Permission reference
- API cheat sheet
- Legal notice

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

WELCOME

What Okapi does for you

Every app that uses the camera, location, photos or notifications has to ask the user first. Doing that well is surprisingly hard. Okapi does it for you.

When your app needs a permission, Okapi runs a complete, friendly flow:

1. **Explains why** on a beautiful screen before the system dialog appears, so users understand and say yes.
2. **Shows the system dialog** (the official Android or iOS pop-up).
3. **Offers to try again** if the user taps "Don't allow".
4. **Guides the user to Settings** if the permission is blocked, then checks again automatically when they come back.

Everything Okapi shows can be restyled, translated, or replaced with your own widgets. Okapi never touches the user's photos, location or any other data: it only manages the permission itself.

How to read this guide

Follow the steps in order. Each step starts with a short summary, then shows exactly what to type or paste. Lines you need to **add** are highlighted in green with a **+** sign, like this:

example.dart

```
1    // Existing line
2  + final okapi = 'added line';
3    // Existing line
```

TIP

Green boxes like this give helpful shortcuts.

WATCH OUT

Orange boxes warn you about common mistakes.

IMPORTANT

Red boxes are things you must not skip.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

BEFORE YOU START

What you need

Ten seconds of checking now saves an hour of confusion later.

- [ ] **Flutter 3.27 or newer.** Check with the command below.
- [ ] **An existing Flutter app** (or create one with `flutter create my_app`).
- [ ] **For Android:** Android Studio with an emulator or a phone with USB debugging.
- [ ] **For iOS:** a Mac with Xcode 16 or newer, and CocoaPods.
- [ ] Your app's minimum versions: **Android 5.0 (API 21)** and **iOS 12** or higher.

Terminal

```
$ flutter --version
Flutter 3.27.0 • channel stable
Dart 3.6.0 • DevTools 2.40.2
```

> **TIP**
>
> If your version is older, run `flutter upgrade` first.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 1

1 Add Okapi to your project

In this step you will: install the package with one command and confirm it worked.

Open a terminal **inside your app's folder** (the folder that contains `pubspec.yaml`) and run:

Terminal

```
$ flutter pub add okapi
Resolving dependencies...
+ okapi 1.0.0
Changed 1 dependency!
Got dependencies!
```

This adds one line to your `pubspec.yaml`. If you prefer, you can add it by hand instead:

pubspec.yaml

```
1    dependencies:
2      flutter:
3        sdk: flutter
4  +   okapi: ^1.0.0
```

Then download it:

Terminal

```
$ flutter pub get
Got dependencies!
```

WATCH OUT

In YAML, spaces matter. `okapi:` must line up exactly under `flutter:` (two spaces from the left). Never use tabs.

STEP 2

2 Set up Android

In this step you will: tell Android which permissions your app uses, and check one build setting.

2.1 Find the Android manifest

Open this file in your editor:

```
my_app/
├── android/
│   └── app/
│       ├── build.gradle   (or build.gradle.kts)
│       └── src/
│           └── main/
│               └── AndroidManifest.xml
├── ios/
├── lib/
└── pubspec.yaml
```

2.2 Add your permissions

Paste one `<uses-permission>` line per permission **above** the `<application>` tag. Only add the ones your app really uses. This example adds camera, location and notifications:

android/app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

+     <uses-permission android:name="android.permission.CAMERA" />
+     <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
+     <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
+     <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

      <application
          android:label="my_app"
          ...
```

IF YOU USE	ADD THESE LINES (NAMES ONLY)
Camera	`android.permission.CAMERA`
Microphone	`android.permission.RECORD_AUDIO`
Photos (Android 13+)	`READ_MEDIA_IMAGES`, `READ_MEDIA_VISUAL_USER_SELECTED`, and `READ_EXTERNAL_STORAGE` with `android:maxSdkVersion="32"`
Location	`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`
Notifications	`POST_NOTIFICATIONS`
Contacts	`READ_CONTACTS`

Bluetooth	`BLUETOOTH_SCAN`, `BLUETOOTH_CONNECT`

The full list is in the Permission reference at the end of this guide.

NOT SURE WHAT TO PASTE?

Okapi can tell you. If you forget a line, `Okapi.ask` stops with an `OkapiException` whose message contains the exact lines to paste. You can also print them yourself:

```
print(OkapiDeclarations.hint(OkapiPermission.photos, isIOS: false));
```

2.3 Check compileSdk

Open `android/app/build.gradle` (or `build.gradle.kts`) and make sure `compileSdk` is **35 or higher**. New projects already use `flutter.compileSdkVersion`, which is fine.

android/app/build.gradle

```
1    android {
2        namespace = "com.example.my_app"
3  +     compileSdk = 35
4        ...
```

IMPORTANT

After changing AndroidManifest.xml, stop the app and run it again (`flutter run`). Hot reload does not pick up manifest changes.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 3

3 Set up iOS

In this step you will: add a short explanation for each permission to Info.plist, then install pods.

3.1 Find Info.plist

```
my_app/
├── android/
├── ios/
│   ├── Podfile
│   └── Runner/
│       └── Info.plist
└── lib/
```

3.2 Add usage descriptions

iOS shows your text inside the system dialog. Add one `<key>` and `<string>` pair per permission, anywhere inside the top-level `<dict>`:

ios/Runner/Info.plist

```
  <plist version="1.0">
  <dict>
+     <key>NSCameraUsageDescription</key>
+     <string>Scan QR codes to pay in one tap.</string>
+     <key>NSLocationWhenInUseUsageDescription</key>
+     <string>Show stores near you.</string>
      <key>CFBundleDisplayName</key>
      <string>My App</string>
      ...
```

IMPORTANT

If a key is missing, iOS **crashes** the app the moment it asks. Okapi checks first and throws a clear `OkapiException` instead, but you still need to add the key.

3.3 Write descriptions Apple accepts

DO	DON'T
"Scan QR codes to pay in one tap."	"This app needs camera access."
"Show stores within 5 km of you."	"Location is required."
Say what the user gets.	Repeat the permission name.

3.4 Install pods

Terminal

```
$ cd ios && pod install && cd ..
Pod installation complete!
```

OPTIONAL: STRIP UNUSED PERMISSION APIS

Apple may question apps that include permission APIs they never use. Tell Okapi to leave them out by adding this to the end of `ios/Podfile` (keep only the flags you want):

IOS/PODFILE

```
post_install do |installer|
  installer.pods_project.targets.each do |target|
    flutter_additional_ios_build_settings(target)
    if target.name == 'okapi'
      target.build_configurations.each do |config|
        config.build_settings['SWIFT_ACTIVE_COMPILATION_CONDITIONS'] =
          '$(inherited) OKAPI_EXCLUDE_CONTACTS OKAPI_EXCLUDE_CALENDAR'
      end
    end
  end
end
```

Available flags: `OKAPI_EXCLUDE_CAMERA`, `_MICROPHONE`, `_PHOTOS`, `_LOCATION`, `_NOTIFICATIONS`, `_CONTACTS`, `_CALENDAR`, `_SPEECH`, `_MEDIA_LIBRARY`, `_TRACKING`, `_SENSORS`, `_BLUETOOTH`. If your Podfile already has a `post_install` block, add only the inner `if` part to it.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 4

4 Import and configure

In this step you will: import Okapi and (optionally) choose its global settings once.

Open `lib/main.dart`. Add the import at the top and one line before `runApp`:

lib/main.dart

```
  import 'package:flutter/material.dart';
+ import 'package:okapi/okapi.dart';

  void main() {
+   Okapi.configure(const OkapiConfig());
    runApp(const MyApp());
  }
```

`OkapiConfig()` with no arguments uses good defaults: a bottom sheet, your app's colours, English text, one retry, and Settings guidance. You can change any of these later (Step 10).

> **TIP**
> Turn on logs while you build: `OkapiConfig(debugLogs: true)` prints every step to the debug console.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 5

5 Ask your first permission

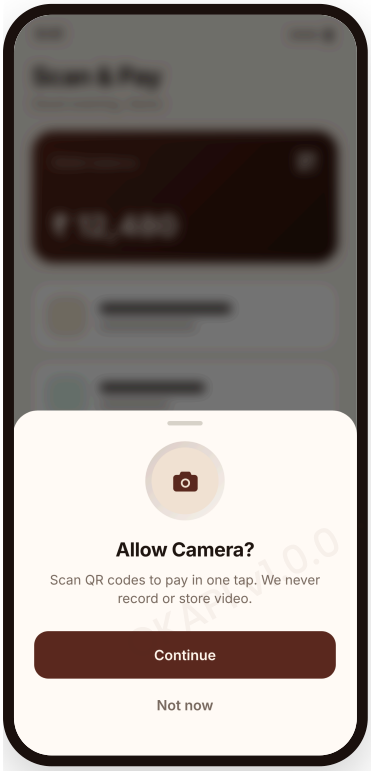
In this step you will: add a button that asks for the camera and see Okapi's flow.

Anywhere you have a `BuildContext` (inside a widget), call `Okapi.ask`:

LIB/SCAN_BUTTON.DART

```dart
ElevatedButton(
  onPressed: () async {
    final result = await Okapi.ask(
      context,
      OkapiPermission.camera,
      reason: 'Scan QR codes to pay in one tap.',
    );

    if (result.isGranted) {
      // The camera can be used now. Open your scanner here.
    }
  },
  child: const Text('Scan QR code'),
)
```



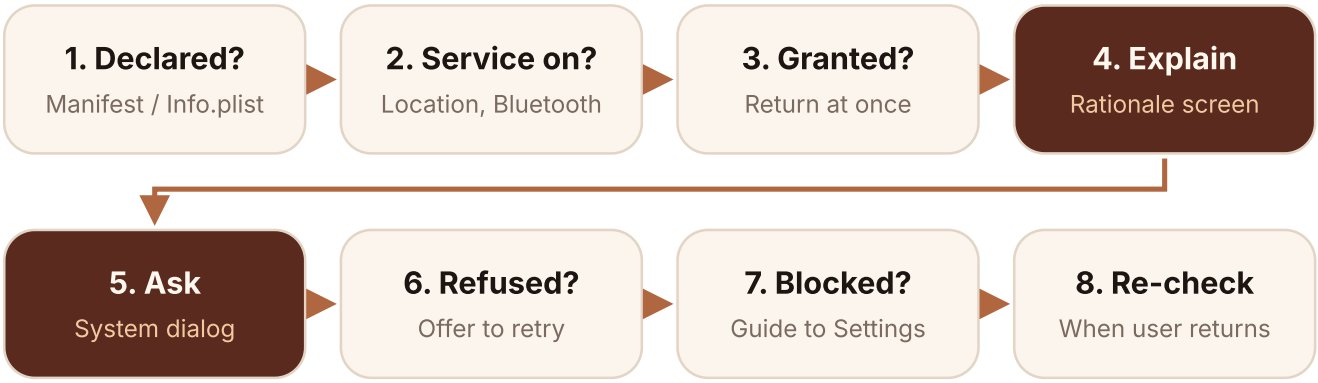
What the user sees first

Here's what happens when the user taps the button:

1. Okapi slides up this sheet explaining *why* (your `reason` text).
2. If they tap **Continue**, the official system dialog appears.
3. If they tap **Not now**, nothing is lost: the system dialog was never used, so you can ask again later.
4. If the camera was already allowed, Okapi skips all of this and returns at once.

This matters because iOS shows its system dialog only once in the app's life. Okapi protects that single chance.

The full flow, at a glance



Every box can be switched off. For example `rationaleMode: OkapiRationaleMode.never` skips the explanation, `maxRetries: 0` skips the retry, `offerSettings: false` skips the Settings guidance.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 6

6 Understand the result

In this step you will: learn what `OkapiResult` tells you and react to each case.

`Okapi.ask` returns an `OkapiResult`. The two fields you'll use most:

- `result.isGranted`: true when the feature can be used now.
- `result.outcome`: *how* the flow ended.

OUTCOME	WHAT HAPPENED	WHAT YOU SHOULD DO
`granted`	Allowed (fully or partly)	Use the feature
`dismissedRationale`	User tapped "Not now" on Okapi's screen	Ask again later, when they tap the feature again
`denied`	User refused in the system dialog	Hide or disable the feature politely
`permanentlyDenied`	Blocked; user didn't go to Settings	Show a small "Enable in Settings" hint
`settingsReturnedDenied`	Went to Settings but didn't switch it on	Respect it; don't nag
`restricted`	Blocked by parental or work controls	Explain it's unavailable on this device
`serviceDisabled`	Location or Bluetooth is switched off	Ask the user to turn it on

REACTING TO THE OUTCOME

```
final result = await Okapi.ask(context, OkapiPermission.location);

switch (result.outcome) {
  case OkapiOutcome.granted:
    showNearbyStores();
  case OkapiOutcome.serviceDisabled:
    showSnack('Please turn on Location Services.');
  default:
    showSnack('You can enable location any time in Settings.');
}
```

Statuses

Behind every result is an `OkapiStatus`. Use `status.canProceed` when you just need "can I use it?".

OKAPISTATUS	MEANING	CANPROCEED
`granted`	Fully allowed	Yes
`limited`	Partly allowed: selected photos only, approximate location, limited contacts	Yes
`provisional`	iOS quiet notifications	Yes
`notApplicable`	Not needed on this phone (e.g. an iOS-only permission on Android)	Yes

`denied`	Not allowed yet; you may ask	No
`permanentlyDenied`	Only Settings can change it	No
`restricted`	Locked by device controls	No

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

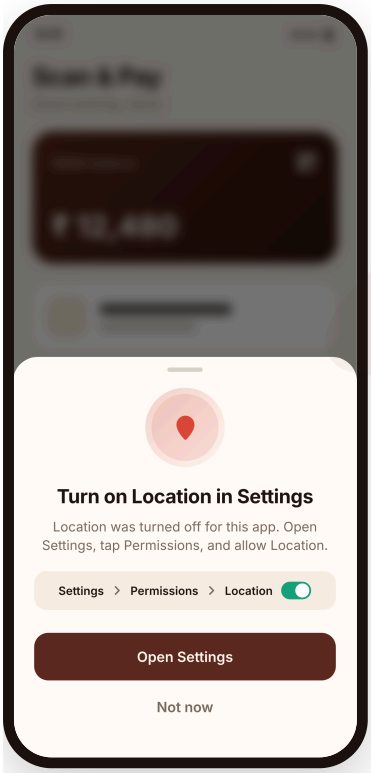
STEP 7

7 When the user says no

In this step you will: see how Okapi handles refusals and blocked permissions.

On **iOS**, the first "Don't Allow" blocks the permission for good. On **Android**, two refusals (or "Don't ask again") do the same. After that, the system dialog will never appear again, and many apps simply break.

Okapi detects this state (`permanentlyDenied`) and, the next time the user taps your feature, shows a clear Settings prompt:



Guiding the user to Settings

1. The red badge signals the permission is off.
2. The little path **Settings › Permissions › Location** shows exactly where to tap.
3. **Open Settings** jumps straight to your app's page in system Settings.
4. When the user comes back, Okapi checks again automatically and your `await` continues with the new result.

GOOD TO KNOW

By default Okapi does **not** show the Settings prompt immediately after the user refuses (that feels pushy). It appears next time. To show it straight away, pass `settingsRightAfterRefusal: true` .

ANDROID DETAIL OKAPI HANDLES FOR YOU

Many packages wrongly report "permanently denied" when the user just closed the Android dialog without choosing. Okapi compares Android's own hint before and after each request to tell the two apart.

Okapi v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 8

8 Ask several permissions

In this step you will: ask for a group, such as camera and microphone for video calls, in one go.

VIDEO CALL PERMISSIONS

```
final results = await Okapi.askAll(
  context,
  [OkapiPermission.camera, OkapiPermission.microphone],
  reasons: {
    OkapiPermission.camera: 'See each other on video calls.',
    OkapiPermission.microphone: 'Hear each other on calls.',
  },
);

final canCall = results.values.every((r) => r.isGranted);
```

Okapi shows **one** explanation listing both permissions, then the system dialogs one after another, and a single Settings prompt for any that are blocked. The results come back in the same order you asked.

NO MORE CRASHES FROM OVERLAPPING REQUESTS

Android crashes if two permission requests run at once. Okapi queues every request automatically, so tapping two buttons quickly is safe.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

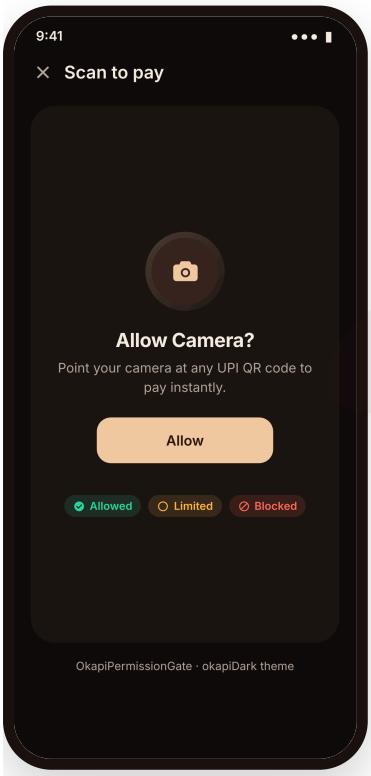
STEP 9

9 Ready-made widgets

In this step you will: drop in widgets that handle permissions for you, with zero logic.

9.1 OkapiPermissionGate

Shows your screen only when the permission is allowed. Otherwise it shows a polished "Allow" card that runs the full flow.



A gate in dark mode

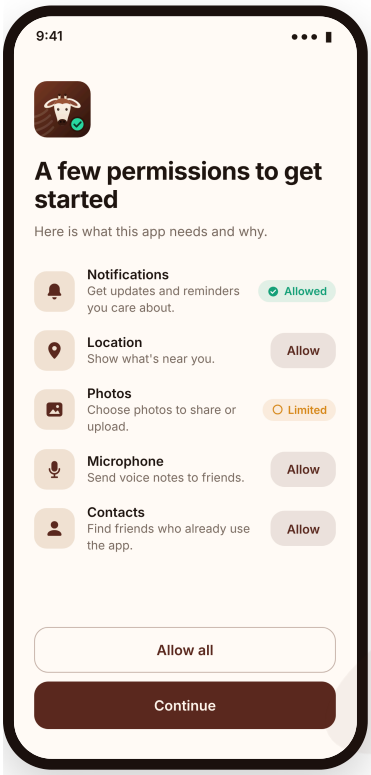
```
OkapiPermissionGate(
  permission: OkapiPermission.camera,
  reason: 'Point your camera at any UPI QR code.',
  child: const QrScannerView(),
)
```

The gate updates by itself if the user changes the permission in Settings and returns. Pass `lockedBuilder` to draw your own locked view, or `autoAsk: true` to start the flow automatically.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

9.2 OkapiOnboarding

A complete "permissions" screen for your app's first launch.



Onboarding screen

```
OkapiOnboarding(
  permissions: const [
    OkapiPermission.notification,
    OkapiPermission.locationWhenInUse,
    OkapiPermission.photos,
  ],
  requiredPermissions: const [
    OkapiPermission.notification,
  ],
  onComplete: (statuses) =>
      Navigator.of(context)
          .pushReplacementNamed('/home'),
)
```

Each row has its own Allow button and live status. **Continue** stays disabled until every permission in `requiredPermissions` is allowed.

9.3 OkapiPermissionTile, OkapiStatusChip, OkapiPermissionBuilder

```
// A settings row with icon, name, reason and an Allow button / status chip.
const OkapiPermissionTile(permission: OkapiPermission.contacts)

// Build anything from the live status.
OkapiPermissionBuilder(
  permission: OkapiPermission.notification,
  builder: (context, status, ask) => SwitchListTile(
    title: const Text('Notifications'),
    value: status?.canProceed ?? false,
    onChanged: (_) => ask(),
  ),
)
```

OKAPI v1.0.0

STEP 10

10 Make it yours

In this step you will: change colours, layout, text and language, or use your own widgets.

10.1 Colours and shapes

Okapi uses your app's colours by default. Choose a preset or adjust anything:

THEME

```
Okapi.configure(OkapiConfig(
  theme: OkapiThemeData.okapiLight().copyWith(
    primary: const Color(0xFF0057FF),   // Button colour
    radius: 20,                         // Sheet corner radius
    buttonRadius: 12,
    blurSigma: 10,                      // Background blur (0 = off)
  ),
));
```

Presets: `OkapiThemeData.fromColorScheme(...)`, `okapiLight()`, `okapiDark()`, `mono()`. For automatic light/dark, add the theme to `ThemeData.extensions` in both `theme` and `darkTheme`. To use an illustration or animation instead of the icon, set `visualBuilder`.

10.2 How prompts appear

PRESENTER	LOOKS LIKE
`OkapiSheetPresenter()`	Bottom sheet with blur (default)
`OkapiDialogPresenter()`	Centred dialog
`OkapiFullScreenPresenter()`	Full page, great for onboarding
`OkapiMixedPresenter(byKind: {...})`	A different style per screen type
`OkapiNoUiPresenter()`	No Okapi screens; logic only
`OkapiCustomPresenter(builder: ...)`	Your own widget (below)

```
Okapi.configure(const OkapiConfig(presenter: OkapiDialogPresenter()));

// Or just for one call:
await Okapi.ask(context, OkapiPermission.contacts,
    presenter: const OkapiFullScreenPresenter());
```

10.3 Your own widget, Okapi's logic

Design the prompt any way you like. Okapi gives your widget a **controller** with everything it needs:

CUSTOM PROMPT WIDGET

```
Okapi.configure(OkapiConfig(
  presenter: OkapiCustomPresenter(
    style: OkapiPresentationStyle.dialog,     // sheet, dialog or fullScreen
    builder: (context, prompt, controller) {
      return Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(controller.iconOf(prompt.permission), size: 56),
          Text(prompt.title, style: const TextStyle(fontSize: 22)),
          Text(prompt.message),
          FilledButton(
            onPressed: controller.accept,         // "Yes" / "Continue"
            child: Text(prompt.primaryLabel),
          ),
          if (prompt.secondaryLabel != null)
            TextButton(
              onPressed: controller.decline,      // "No" / "Not now"
              child: Text(prompt.secondaryLabel!),
            ),
        ],
      );
    },
  ),
));
```

CONTROLLER MEMBER	WHAT IT DOES
`accept()`	Primary action; Okapi continues the flow
`decline()`	Secondary action or dismiss
`openSettings()`	Opens your app's Settings page
`status(permission)`	Reads a live status
`nameOf()`, `iconOf()`	Localised name and Okapi's icon
`prompt.kind`	Which screen: rationale, denied, settings, serviceDisabled, restricted, multiRationale
`theme`, `strings`	Okapi's resolved theme and text

CUSTOMISE ONLY SOME SCREENS

Use `builders: {OkapiPromptKind.settings: mySettingsBuilder}` ; other screens keep Okapi's design. Set `wrapInFrame: false` to draw the whole container yourself.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

10.4 Text and languages

Okapi ships English, Hindi and Marathi. Change any sentence without touching the rest:

```
// Hindi everywhere:
Okapi.configure(const OkapiConfig(strings: OkapiStrings.hindi));

// Or override just a few texts:
Okapi.configure(OkapiConfig(
  strings: const OkapiStrings(
    rationaleAllow: 'Sounds good',
    permissionReasons: {
      OkapiPermission.camera: 'Scan QR codes to pay.',
    },
  ),
  // Last-minute edit of any prompt:
  promptOverride: (p) => p.kind == OkapiPromptKind.settings
      ? p.copyWith(title: 'One quick switch in Settings')
      : p,
));
```

Templates can use `{permission}`, `{reason}` and `{service}`; Okapi fills them in.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 11

11 Analytics hooks

In this step you will: measure how users respond, using your own analytics.

```
class PermissionAnalytics extends OkapiObserver {
  @override
  void onResult(OkapiResult r) {
    myAnalytics.log('permission_${r.permission.name}', r.outcome.name);
  }

  @override
  void onStatusChange(OkapiStatusChange c) {
    // Fires even when the user changes it in system Settings.
  }
}

Okapi.configure(OkapiConfig(observers: [PermissionAnalytics()]));
```

PRIVACY

Okapi itself never sends data anywhere. These hooks run only inside your app; if you forward events to an analytics service, that's your app's data processing and must appear in your privacy policy.

Live updates

```
Okapi.watch(OkapiPermission.notification).listen((status) {
  // Current status first, then every change.
});
```

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 12

12 Test it

In this step you will: test your permission logic without a phone.

TEST/PERMISSIONS_TEST.DART

```dart
import 'package:flutter_test/flutter_test.dart';
import 'package:okapi/okapi.dart';
import 'package:okapi/testing.dart';

void main() {
  late FakeOkapiPlatform fake;

  setUp(() {
    fake = FakeOkapiPlatform(
      statuses: {OkapiPermission.camera: OkapiStatus.denied},
      requestResults: {OkapiPermission.camera: OkapiStatus.granted},
    );
    OkapiPlatform.instance = fake;
    Okapi.configure(const OkapiConfig(presenter: OkapiNoUiPresenter()));
  });

  test('camera request', () async {
    expect(await Okapi.request(OkapiPermission.camera), OkapiStatus.granted);
    expect(fake.requests.length, 1);
  });
}
```

On real devices, test each permission at least three ways: allow, refuse, and refuse then enable in Settings. On Android 11+, also test refusing twice.

RESET A PERMISSION ON A DEVICE

Android: `adb shell pm reset-permissions` or uninstall the app. iOS: Settings › General › Transfer or Reset › Reset Location & Privacy, or delete the app.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 13

13 Check and fix with Doctor and Sync

In this step you will: let Okapi check your setup and write missing declarations for you.

Okapi ships two small command-line tools. Run them in a terminal inside your app's folder. They only read your code and edit your platform files; they never send anything anywhere.

13.1 Okapi Doctor: find problems

Terminal

```
$ dart run okapi:doctor

Okapi Doctor · my_app

Code
  ✓ 3 permissions used: camera, locationWhenInUse, notification

Android
  ✗ camera: missing android.permission.CAMERA
      <uses-permission android:name="android.permission.CAMERA" />

iOS
  ✗ camera: missing NSCameraUsageDescription in Info.plist

App Store
  ! NSLocationWhenInUseUsageDescription sounds generic ("needs access")

2 errors, 1 warning, 0 notes
Fix declarations automatically: dart run okapi:sync
```

DOCTOR CHECKS	WHY IT MATTERS
Permissions your code uses but didn't declare	Android silently fails; iOS crashes the app
Permissions declared but never asked for	Store reviewers ask why you need them
Google Play restricted permissions	SMS, call log, all files, background location and others need a Play Console declaration
Vague or placeholder iOS texts	Apple rejects reasons like "This app needs access"
Unused iOS permission APIs compiled in	Avoids App Store questions
compileSdk and iOS version	Okapi needs compileSdk 35 and iOS 12+

13.2 Okapi Sync: fix them

Preview first, then apply:

Terminal

```
$ dart run okapi:sync --dry-run
$ dart run okapi:sync

Okapi Sync · my_app
  ✓ Permissions used: camera, locationWhenInUse, notification
  i Updated android/app/src/main/AndroidManifest.xml
  i Updated ios/Runner/Info.plist
      Replace each TODO text in Info.plist with a real reason before release.

2 files updated. Next: dart run okapi:doctor
```

Sync writes only inside a marked region, so your own lines are never touched:

android/app/src/main/AndroidManifest.xml

```
1    <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2  +     <!-- okapi:begin (managed by `dart run okapi:sync`; edits inside are replaced) -->
3  +     <uses-permission android:name="android.permission.CAMERA" />
4  +     <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
5  +     <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
6  +     <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
7  +     <!-- okapi:end -->
8        <application
```

IMPORTANT

In Info.plist, Sync adds texts that start with `TODO:` . Replace each one with a real reason in your own words. Doctor reports them as errors until you do. Sync keeps the texts you edit.

PERMISSIONS CHOSEN AT RUNTIME

If your code picks permissions with `OkapiPermission.values` or `.byName` , the tools can't see which ones. List them in `pubspec.yaml` :

PUBSPEC.YAML

```
okapi:
  permissions: [contacts, calendarFullAccess]
  ios_keep: [BLUETOOTH]
```

STEP 14

14 Ask for features

In this step you will: ask for everything a feature needs at once and learn how much of it can run.

Describe each feature once, near the top of a file:

LIB/FEATURES.DART

```
const videoCall = OkapiFeature(
  id: 'video_call',
  name: 'Video calls',
  description: 'See and hear your friends.',
  required: [OkapiPermission.camera, OkapiPermission.microphone],
  optional: [OkapiPermission.bluetoothConnect],   // headset
  reasons: {
    OkapiPermission.camera: 'So friends can see you.',
    OkapiPermission.microphone: 'So friends can hear you.',
  },
);
```

Then ask for the feature. Okapi shows one explanation with an **Optional** tag on optional items:

```
final r = await Okapi.askFeature(context, videoCall);

switch (r.level) {
  case OkapiFeatureLevel.full:
    startCall();                      // everything allowed
  case OkapiFeatureLevel.partial:
    startCall(headset: false);        // optional item refused
  case OkapiFeatureLevel.unavailable:
    showMessage('Video calls need the camera and microphone.');
}
```

Or let a widget decide what to show:

```
OkapiFeatureGate(
  feature: videoCall,
  child: const CallScreen(),
  partialChild: const CallScreen(headset: false),
)
```

STEP 15

15 Stop nagging with an ask policy

In this step you will: set friendly limits so users aren't asked again and again.

LIB/MAIN.DART

```
Okapi.configure(const OkapiConfig(
  askPolicy: OkapiAskPolicy.recommended,
));
```

RULE IN `RECOMMENDED`	VALUE
Skip the very first app launch	1 launch
Wait after "Not now" on Okapi's screen	2 days
Wait after the user refused in the system dialog	7 days
Wait after the user declined to open Settings	14 days
Okapi prompts per app session	2

When a rule applies, no UI is shown and the result's outcome is `OkapiOutcome.deferredByPolicy`. `result.policy.reason` says which rule. Change any value with `OkapiAskPolicy.recommended.copyWith(cooldownAfterNotNow: Duration(days: 1))`.

WHEN THE USER TAPS A SWITCH

If the user explicitly asked (for example tapped "Enable notifications" in your settings), pass `force: true` so the policy never blocks them.

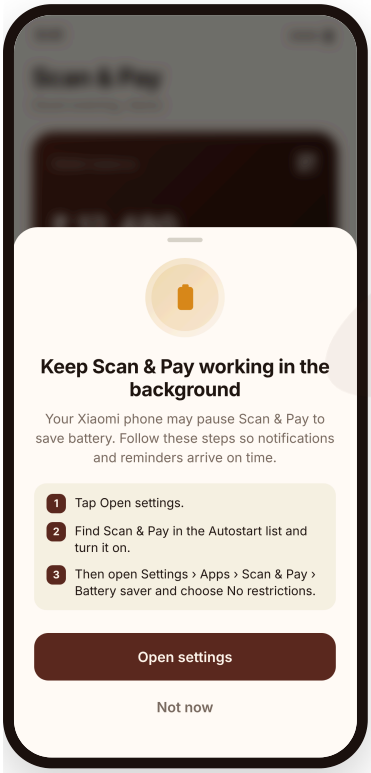
OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 16

16 Keep apps alive on Indian phones

In this step you will: make reminders and notifications reliable on Xiaomi, vivo, OPPO, Samsung and other phones.

Many phones popular in India stop apps in the background to save battery. Notifications arrive late, alarms don't ring, and users blame your app. Okapi detects the phone maker, shows the exact steps and opens the right hidden screen.



The guide on a Xiaomi phone

```dart
final r = await Okapi.askBackgroundReliability(
  context,
  reason: 'So your medicine reminders '
      'ring on time.',
);

if (r.ready) scheduleReminders();
```

1. Okapi shows brand-specific steps (here, Xiaomi).
2. **Open settings** opens the phone's Autostart screen.
3. When the user comes back, Okapi asks "Did you turn it on?" because phones don't let apps read this setting.
4. Then it handles battery optimisation.

PHONE MAKER	WHAT OKAPI OPENS
Xiaomi, Redmi, POCO	Autostart list, then Battery saver › No restrictions
vivo, iQOO	Background start / Autostart manager
OPPO, realme, OnePlus	Auto launch list
Samsung	Battery › Background usage limits
TECNO, Infinix, itel	Phone Master › Auto start
Huawei, Honor, ASUS, Nokia	App launch / Auto-start / battery protection
Other Android phones	Android's Battery optimisation list

Developed by Sanket Santosh Vidya Manohar · Okapi v1.0.0

ASK AT THE RIGHT MOMENT

Show the guide when it matters (after the user creates their first reminder), not at app launch. It does nothing on iPhones or when everything is already set up.

OPTIONAL BATTERY PERMISSION

If your app qualifies under Google Play's policy, also declare `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS` so Okapi can ask Android directly. Doctor reminds you about the Play policy.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 17

17 Protect from auto-reset

In this step you will: stop Android from removing permissions when your app isn't used for a while.

Android removes permissions from apps that aren't opened for a few months. Okapi remembers what was allowed and notices when something disappears. The gentlest approach is to ask only when that actually happened:

```
// For example on your home screen, after the first frame:
await Okapi.protectFromAutoReset(context, onlyIfRevoked: true);

// Or check yourself:
final removed = await Okapi.revokedSinceLastLaunch();
final status = await Okapi.autoResetStatus();   // enabled, exempt, notAvailable
```

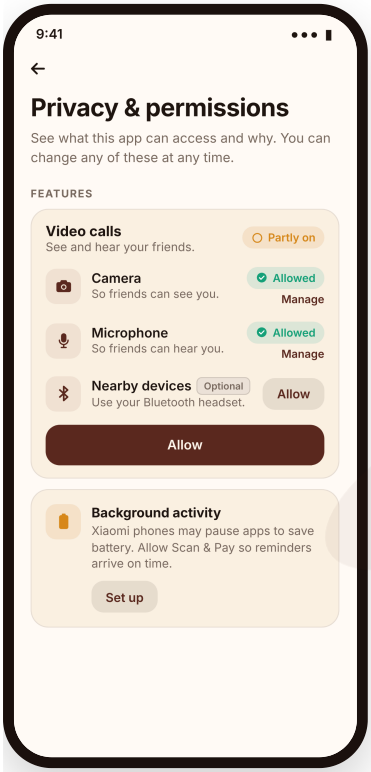
The prompt names the removed permissions and opens the exact screen where the user turns off **Pause app activity if unused**. It does nothing on iPhones.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 18

18 Add a Privacy Center

In this step you will: give users one clear screen to see and change what your app can access.



Privacy Center

```
ListTile(
  title: const Text('Privacy & permissions'),
  onTap: () => Navigator.of(context).push(
    MaterialPageRoute(
      builder: (_) => OkapiPrivacyCenter(
        features: [videoCall],
        permissions: [
          OkapiPermission.notification,
        ],
        onOpenPrivacyPolicy: openPolicy,
      ),
    ),
  ),
)
```

Every row updates live. Users can allow what's off and tap **Manage** to change what's on. On Android, cards for background activity and unused-app protection appear when relevant.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

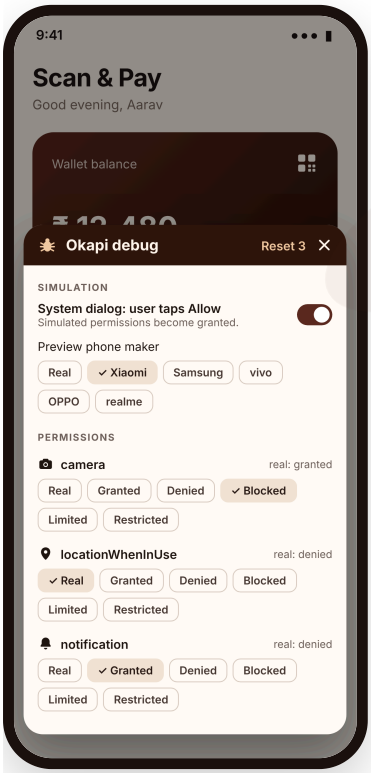
STEP 19

19 Test with the debug panel

In this step you will: test every permission case on a real phone in seconds.

lib/main.dart

```
  MaterialApp(
+   builder: (context, child) => OkapiDebugPanel(child: child!),
    home: const HomePage(),
  )
```



The debug panel

A small floating button appears in debug builds. Tap it to:

1. Set any permission to Granted, Denied, Blocked, Limited or Restricted.
2. Make the system dialog "tap Allow".
3. Switch Location or Bluetooth off.
4. Preview the background guide for Xiaomi, Samsung, vivo, OPPO and more.
5. Simulate auto-reset and clear the ask history.

No reinstalling and no phone resets. The panel is never active in release builds, so you can leave it in your code.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

STEP 20

20 Before you publish

In this step you will: go through a final checklist so stores approve your app.

Both platforms

- [ ] `dart run okapi:doctor` shows 0 errors.
- [ ] Every `TODO:` text from Okapi Sync is replaced with a real reason.
- [ ] Every permission you request has a clear, honest `reason` in your own words.
- [ ] You ask at the moment the user needs the feature, not all at launch.
- [ ] Your app works (with reduced features) when a permission is refused.
- [ ] Your privacy policy explains what your app does with the data each permission gives it.

Android / Google Play

- [ ] The manifest lists only the permissions you use.
- [ ] Play Console's **Data safety** form describes your app's data use.
- [ ] Restricted permissions (SMS, call log, all files access, background location, exact alarms, install packages) have the declarations Play requires, or are removed.
- [ ] Tested on Android 10, 12, 13 and 14+ if possible.

iOS / App Store

- [ ] Every used permission has a specific usage description in Info.plist.
- [ ] Unused permission families are excluded with Podfile flags (Step 3.4).
- [ ] App Store Connect's **App Privacy** answers match your app.
- [ ] If you use tracking, the ATT prompt appears before any tracking happens.

> **OKAPI'S OWN PRIVACY**
>
> Okapi collects nothing and adds nothing to your store privacy forms. It includes an Apple privacy manifest declaring this.

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

HELP

Troubleshooting

The most common problems and how to fix them in a minute.

WHAT YOU SEE	WHY, AND THE FIX
`OkapiException(not_declared)`	The permission is missing from AndroidManifest.xml or Info.plist. Paste the lines from the error message, then stop and re-run the app (not hot reload).
`no_activity` on Android	You asked before the app was on screen. Ask from a visible widget, for example after a button tap.
Always `permanentlyDenied` on iOS after one refusal	That's how iOS works: one refusal blocks it. Okapi guides the user to Settings.
`storage` returns `notApplicable`	Android 13+ replaced storage with `photos`, `videos` and `audio`. Ask for those.
Background location never asked	Android needs foreground location first. Just ask `locationAlways`; Okapi asks in the right order.
Tracking (ATT) dialog doesn't appear	Add `NSUserTrackingUsageDescription`. Okapi already waits until the app is active. Also check Settings › Privacy › Tracking is allowed on the test phone.
Notifications always granted on old Android	Before Android 13 there's no dialog; notifications are on unless the user turned them off.
Prompt didn't appear at all	The permission was already allowed (Okapi returns immediately), or you passed `OkapiNoUiPresenter`. Turn on `debugLogs` to see each step.
Build error in ios after adding flags	Run `cd ios && pod install` again and clean the build folder in Xcode.
Outcome is `deferredByPolicy`	Your ask policy postponed it (see `result.policy.reason`). Pass `force: true` when the user explicitly asked, or use "Reset ask history" in the debug panel.
Background guide doesn't open a screen	That phone hides the screen or uses a different name. Okapi falls back to Android's battery list; the steps still guide the user.
Doctor says permissions are chosen at runtime	List them under `okapi: permissions:` in pubspec.yaml.
Sync left the Podfile alone	You already added `OKAPI_EXCLUDE` flags by hand. Remove them to let Sync manage them, or keep managing them yourself.

REFERENCE

Permission reference

All 37 permissions. Android names are shown without the `android.permission.` prefix. "n/a" returns `notApplicable`; "special" opens a system screen and Okapi re-checks when the user returns.

OKAPIPERMISSION	ANDROID MANIFEST	IOS INFO.PLIST
`camera`	CAMERA	NSCameraUsageDescription
`microphone`	RECORD_AUDIO	NSMicrophoneUsageDescription
`speech`	RECORD_AUDIO	NSSpeechRecognitionUsageDescription
`photos`	READ_MEDIA_IMAGES, READ_MEDIA_VISUAL_USER_SELECTED, READ_EXTERNAL_STORAGE (max 32)	NSPhotoLibraryUsageDescription
`photosAddOnly`	n/a	NSPhotoLibraryAddUsageDescription
`videos`	READ_MEDIA_VIDEO, READ_EXTERNAL_STORAGE (max 32)	NSPhotoLibraryUsageDescription
`audio`	READ_MEDIA_AUDIO, READ_EXTERNAL_STORAGE (max 32)	n/a
`storage`	READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE (max 28)	n/a
`manageExternalStorage`	MANAGE_EXTERNAL_STORAGE (special)	n/a
`mediaLocation`	ACCESS_MEDIA_LOCATION	n/a
`mediaLibrary`	n/a	NSAppleMusicUsageDescription
`location` / `locationWhenInUse`	ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION	NSLocationWhenInUseUsageDescription
`locationAlways`	+ ACCESS_BACKGROUND_LOCATION	+ NSLocationAlwaysAndWhenInUseUsageDescription
`notification`	POST_NOTIFICATIONS	none
`criticalAlerts`	n/a	none (Apple entitlement)
`accessNotificationPolicy`	ACCESS_NOTIFICATION_POLICY (special)	n/a
`contacts`	READ_CONTACTS, WRITE_CONTACTS	NSContactsUsageDescription
`calendarFullAccess`	READ_CALENDAR, WRITE_CALENDAR	NSCalendarsFullAccessUsageDescription
`calendarWriteOnly`	WRITE_CALENDAR	NSCalendarsWriteOnlyAccessUsageDescription
`reminders`	n/a	NSRemindersFullAccessUsageDescription
`phone`	READ_PHONE_STATE, CALL_PHONE, ... (any)	n/a
`sms`	SEND_SMS, READ_SMS, ... (any)	n/a

`sensors`	BODY_SENSORS	NSMotionUsageDescription
`sensorsAlways`	+ BODY_SENSORS_BACKGROUND	n/a
`activityRecognition`	ACTIVITY_RECOGNITION	NSMotionUsageDescription
`bluetooth`	BLUETOOTH_SCAN, BLUETOOTH_CONNECT	NSBluetoothAlwaysUsageDescription
`bluetoothScan / Connect / Advertise`	matching BLUETOOTH_*	NSBluetoothAlwaysUsageDescription
`nearbyWifiDevices`	NEARBY_WIFI_DEVICES	n/a
`appTrackingTransparency`	n/a	NSUserTrackingUsageDescription
`ignoreBatteryOptimizations`	REQUEST_IGNORE_BATTERY_OPTIMIZATIONS (special)	n/a
`systemAlertWindow`	SYSTEM_ALERT_WINDOW (special)	n/a
`requestInstallPackages`	REQUEST_INSTALL_PACKAGES (special)	n/a
`scheduleExactAlarm`	SCHEDULE_EXACT_ALARM or USE_EXACT_ALARM (special)	n/a
`backgroundRefresh`	n/a	none (read-only)

OKAPI v1.0.0 · Developed by Sanket Santosh Vidya Manohar

REFERENCE

API cheat sheet

Everything on one page.

CHEAT SHEET

```
// Guided flows
Okapi.ask(context, permission, {reason, presenter, rationaleMode, maxRetries,
          offerSettings, settingsRightAfterRefusal, checkService, provisional})
Okapi.askAll(context, permissions, {reasons, presenter, rationaleMode, offerSettings})

// Low level (no Okapi UI)
Okapi.status(permission)              Okapi.statuses(permissions)
Okapi.request(permission)             Okapi.requestMany(permissions)
Okapi.shouldShowRationale(permission) Okapi.serviceStatus(permission)
Okapi.openAppSettings()               Okapi.openServiceSettings(permission)
Okapi.isDeclared(permission)          Okapi.assertDeclared(permission)

// Features, policy, reliability
Okapi.askFeature(context, feature)    Okapi.featureStatus(feature)
OkapiConfig(askPolicy: OkapiAskPolicy.recommended)   ask(..., force: true)
Okapi.askBackgroundReliability(context, {reason, force})
Okapi.backgroundStatus()              Okapi.needsBackgroundSetup()
Okapi.deviceInfo()  Okapi.oemScreens()  Okapi.openOemScreen(screen)
Okapi.protectFromAutoReset(context, {onlyIfRevoked, force})
Okapi.autoResetStatus()               Okapi.revokedSinceLastLaunch()
Okapi.watch(permission)               Okapi.statusChanges
Okapi.lastKnown(permission)           Okapi.refreshKnown()

// Configuration
Okapi.configure(OkapiConfig(presenter, theme, strings, rationaleMode, maxRetries,
  offerSettings, settingsRightAfterRefusal, checkService, settingsReturnTimeout,
  observers, promptOverride, debugLogs))

// Widgets
OkapiPermissionGate  OkapiPermissionBuilder  OkapiPermissionTile
OkapiStatusChip      OkapiOnboarding         OkapiLockedCard
OkapiFeatureGate     OkapiFeatureBuilder     OkapiPrivacyCenter
OkapiDebugPanel      OkapiStepList           OkapiTag
OkapiPromptView      OkapiIconBadge

// Presenters
OkapiSheetPresenter  OkapiDialogPresenter  OkapiFullScreenPresenter
OkapiMixedPresenter  OkapiNoUiPresenter    OkapiFunctionPresenter
OkapiCustomPresenter(builder | builders, style, wrapInFrame, fallback)

// Testing
import 'package:okapi/testing.dart';  // FakeOkapiPlatform, fakeAndroidDevice

// Command line
dart run okapi:doctor [--json] [--strict]
dart run okapi:sync [--dry-run] [--with-optional] [--no-podfile]
```

LEGAL

Legal notice

A short summary. The full documents ship with the package and always prevail over this summary.

License

Okapi is licensed under the **BSD 3-Clause license**, file `LICENSE`. You may use Okapi free of charge in any application, including commercial applications. If you redistribute the source code, keep the copyright notice and license text. Copyright © 2026 Sanket Santosh Vidya Manohar.

Name and logo

The BSD license covers the code only. "Okapi" and the Okapi logo are trademarks of Sanket Santosh Vidya Manohar. A fork or modified copy must be published under a different name and logo and must not claim to be the official Okapi (see `TRADEMARKS.md`).

A development tool, not a decision-maker

Okapi is strictly a software development package. It does not decide, recommend or approve whether, when or why any permission is requested: every request comes from the developer's own code. Okapi never accesses, collects, stores or transmits photos, locations, contacts, recordings or any other user data, and makes no network requests (see `PRIVACY.md`).

No responsibility for permissions or data breaches

The Author is not responsible and shall not be liable for any permission requested, granted or used through Okapi, or for any data breach, data leak, unauthorised access or misuse of data by any app that uses Okapi, including data an app could access because a permission was granted. The app that requests a permission, and the person or company behind it, are solely responsible for that decision, for consent, for the truthfulness of every explanation shown, for the security of the data, for breach notification, and for compliance with laws (including India's Digital Personal Data Protection Act, 2023, GDPR and CCPA) and with Google Play and App Store policies.

No guarantee

Okapi is provided **"as is"**, without warranty of any kind and without any guarantee of correct results. Permission behaviour differs between devices, manufacturers and OS versions. Phone-maker screens and steps may change or not exist on a given phone, and Okapi Doctor and Okapi Sync are automated helpers, not legal or policy advice: review every change they make. To the maximum extent permitted by law, the Author is not liable for any loss or damage, and total liability is nil.

Full terms

Binding terms: `LICENSE`, `TERMS.md`, `DISCLAIMER.md`, `PRIVACY.md` and `TRADEMARKS.md`. They prevail over this summary.



Thank you for building with Okapi.

Okapi v1.0.0 · Developed by Sanket Santosh Vidya Manohar

sanketmanohar.com · github.com/sanket-manohar/okapi · pub.dev/publishers/sanketmanohar.com